Distributed Systems Overview: Foundations, Challenges & the CAP Theorem

Objective

The objective of this training is to give individuals the foundational knowledge needed to reason about distributed systems: what makes a system distributed, the core challenges that come with it, the CAP theorem, and the fundamentals of data partitioning and replication. It is written to be generic so any group or team can adopt it, regardless of skill level (beginner, intermediate, or advanced).

Outline

- Introduction & Purpose
  - Why Distributed Systems Knowledge Matters
  - Scope: Who This Applies To
- What Is a Distributed System
  - Definition & Everyday Examples
  - Key Characteristics
- Core Challenges in Distributed Systems
  - Network Unreliability
  - Partial Failures
  - Consistency vs Availability Trade-offs
  - Ordering, Clocks & Time
- The CAP Theorem
  - Consistency, Availability, Partition Tolerance Defined
  - Why You Can Only Guarantee Two Out of Three
  - CP vs AP Systems: Real-World Examples
  - Beyond CAP: The PACELC Extension
- Data Distribution: Partitioning & Consistent Hashing
  - Partitioning Strategies
  - The Problem With Naive Hashing
  - How Consistent Hashing Works
  - Virtual Nodes & Even Load Distribution
- Replication Fundamentals
  - Why Replicate Data
  - Synchronous vs Asynchronous Replication
  - Leader-Follower vs Multi-Leader vs Leaderless
  - Replication Trade-offs Through the CAP Lens
- Common Pitfalls & Case Studies
  - Anonymized Example Failure Scenarios
  - Lessons Learned
  - Indexing vs. Replicas vs. Vertical Scaling/Splitting vs. Horizontal Sharding
- Additional Resources
  - Public Tutorials & References

Introduction & Purpose

Nearly every application you build today, whether it serves ten users or ten million, eventually runs on more than one machine. Understanding why systems are distributed, what breaks when they are, and the vocabulary used to reason about those trade-offs is foundational knowledge for any developer, not just infrastructure specialists. This document builds that foundation from first principles, then connects it to two concrete topics you will use in practice: data partitioning and replication.

Why Distributed Systems Knowledge Matters

- Even a simple web app is a distributed system once it has a separate database, a cache, a load balancer, or runs on more than one server instance.
- Bugs in distributed systems are often intermittent and hard to reproduce, because they depend on timing, network conditions, or partial failures that do not show up on a single laptop.
- Design decisions made early, such as how data is partitioned or replicated, are expensive to change later. Understanding the trade-offs up front avoids costly rewrites.
- Design-review and interview conversations at almost every architecture touch these concepts, so fluency here is broadly useful.

Scope: Who This Applies To

This material is written to be useful at every experience level. Depending on your background, focus your first pass differently:

- Beginner: New to distributed systems concepts. Focus on What Is a Distributed System and Core Challenges to build vocabulary and intuition before anything else.
- Intermediate: Comfortable with basic client-server systems, ready to reason about trade-offs. Focus on The CAP Theorem through Replication Fundamentals.
- Advanced: Designing or reviewing data-layer architecture. Focus on Data Distribution through Common Pitfalls & Case Studies, and use Additional Resources to go deeper on specific algorithms.

What Is a Distributed System

Definition & Everyday Examples

A distributed system is a collection of independent computers that appears to its users as a single coherent system. The computers communicate and coordinate only by passing messages over a network; there is no shared memory and no shared clock.

- A web application with a separate application server and database server is already a distributed system, even with just two machines.
- A content delivery network serving the same image from data centers on different continents is a distributed system operating at global scale.
- A mobile app that syncs data with a cloud backend while occasionally offline is also a distributed system, with the network link being the least reliable part.

Key Characteristics

Check Item	What It Means in Practice
Concurrency	Multiple nodes execute independently and simultaneously; there is no single thread of control to reason about.
No global clock	Each machine has its own clock, and clocks drift. Nodes cannot simply compare local timestamps to agree on order.
Independent failure	Any single node, disk, or network link can fail while the rest of the system keeps running. Failures are partial, not all-or-nothing.
Scalability by design	Capacity is added by adding more machines rather than making one machine bigger, which introduces coordination overhead.

Core Challenges in Distributed Systems

Network Unreliability

Unlike a function call within a single process, a network call can be slow, can be lost, or can succeed on the server while the response back to the client is lost. The caller often cannot tell the difference between these cases, which shapes how retries and idempotency must be designed.

- Latency varies unpredictably, so systems must set sensible timeouts rather than assume a fixed response time.
- Network partitions can temporarily split a cluster into groups that cannot reach each other, even though every individual node is healthy.

Partial Failures

In a single-machine program, a crash usually stops everything at once. In a distributed system, one node can fail while others continue serving traffic. Handling this correctly, detecting the failure, redirecting traffic, and eventually recovering, is one of the central engineering problems in this space.

Consistency vs Availability Trade-offs

When one part of the system cannot communicate with another part, a design decision has to be made: keep serving requests and risk returning stale or conflicting data, or refuse to serve some requests until the system can guarantee correctness. Neither choice is universally right; the correct answer depends on what the data is used for.

Example:

Consistency: An ATM must reject cash withdrawals during a network outage to prevent double-spending from the same account balance.

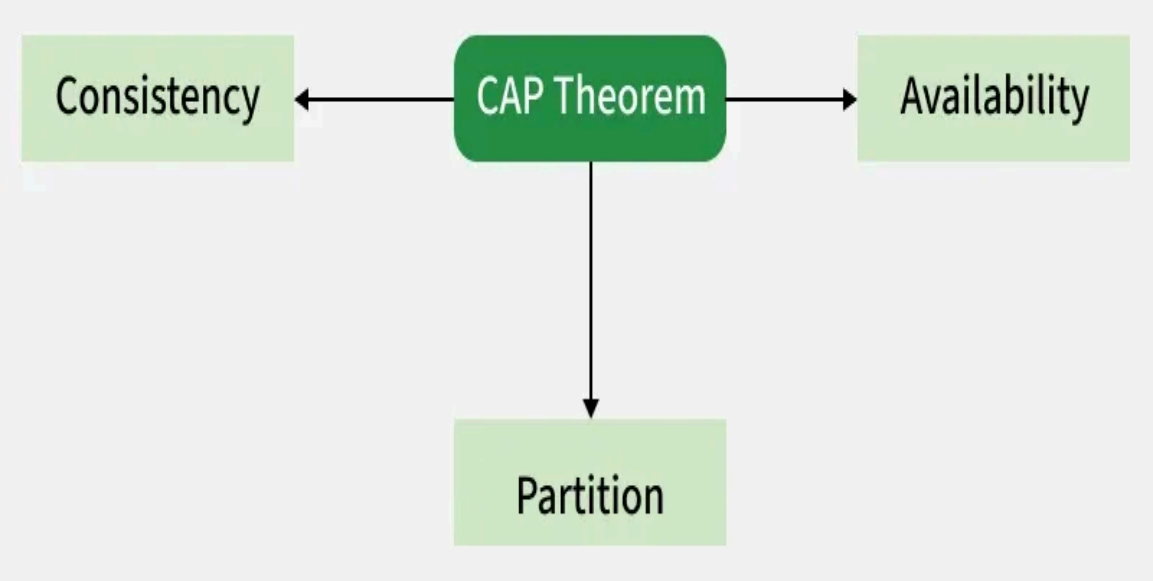
Availability: An e-commerce site still lets you add items to your cart during a server disruption, syncing and resolving duplicates later at checkout.

Ordering, Clocks & Time

Because there is no shared clock, distributed systems often cannot rely on wall-clock timestamps to decide which of two events happened first. Techniques such as logical clocks and vector clocks exist specifically to establish a reliable notion of "before" and "after" without depending on synchronized physical clocks.

The CAP Theorem

Consistency, Availability, Partition Tolerance Defined

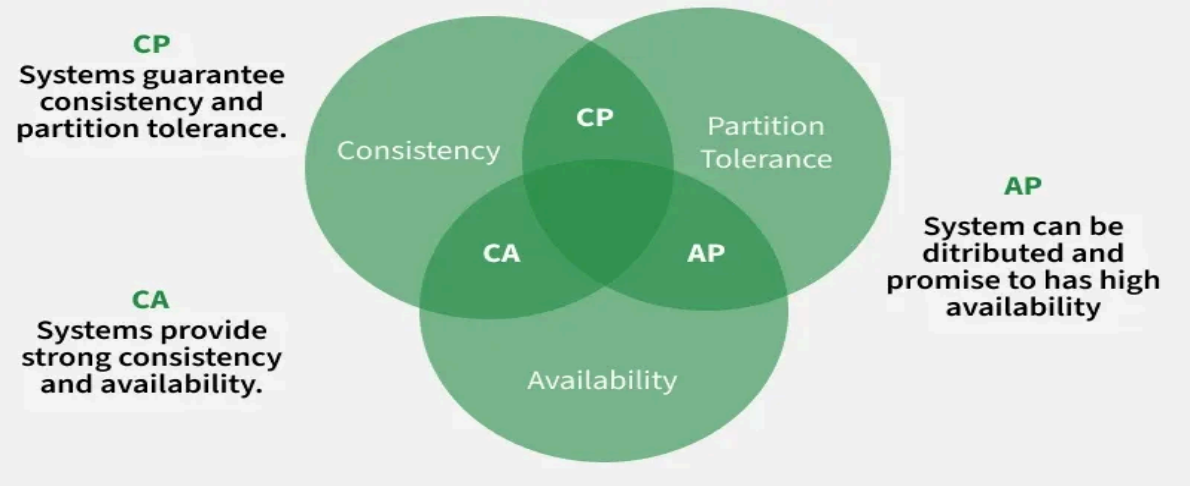


Check Item		Definition
Consistency (C)		Every read receives the most recent write, or an error. All nodes see the same data at the same time.

Availability (A)		Every request receives a non-error response, without the guarantee that it contains the most recent write.
Partition Tolerance (P)		The system continues to operate despite an arbitrary number of messages being dropped or delayed between nodes.

Why You Can Only Guarantee Two Out of Three

Network partitions will happen; they are a fact of operating over real networks, so partition tolerance is effectively mandatory for any distributed system. The real decision the CAP theorem forces is what happens during a partition: does the system favor consistency (some nodes refuse to answer to avoid returning stale data) or availability (every node keeps answering, accepting the risk of returning inconsistent data)?



CP vs AP Systems: Real-World Examples

- CP systems, such as many configuration and coordination stores, prefer to reject requests during a partition rather than risk returning incorrect data, since correctness there matters more than uptime. Example - Banking system.
- AP systems, such as many shopping-cart or session-store use cases, prefer to keep serving requests during a partition and reconcile any conflicts once the network heals, since availability matters more than perfect consistency for that data.

Beyond CAP: The PACELC Extension

CAP only describes behavior during a network partition. PACELC extends the idea: even when there is no partition (Else), a system still has to trade off Latency against Consistency, because waiting to confirm data is fully replicated takes time. This is a useful mental model when

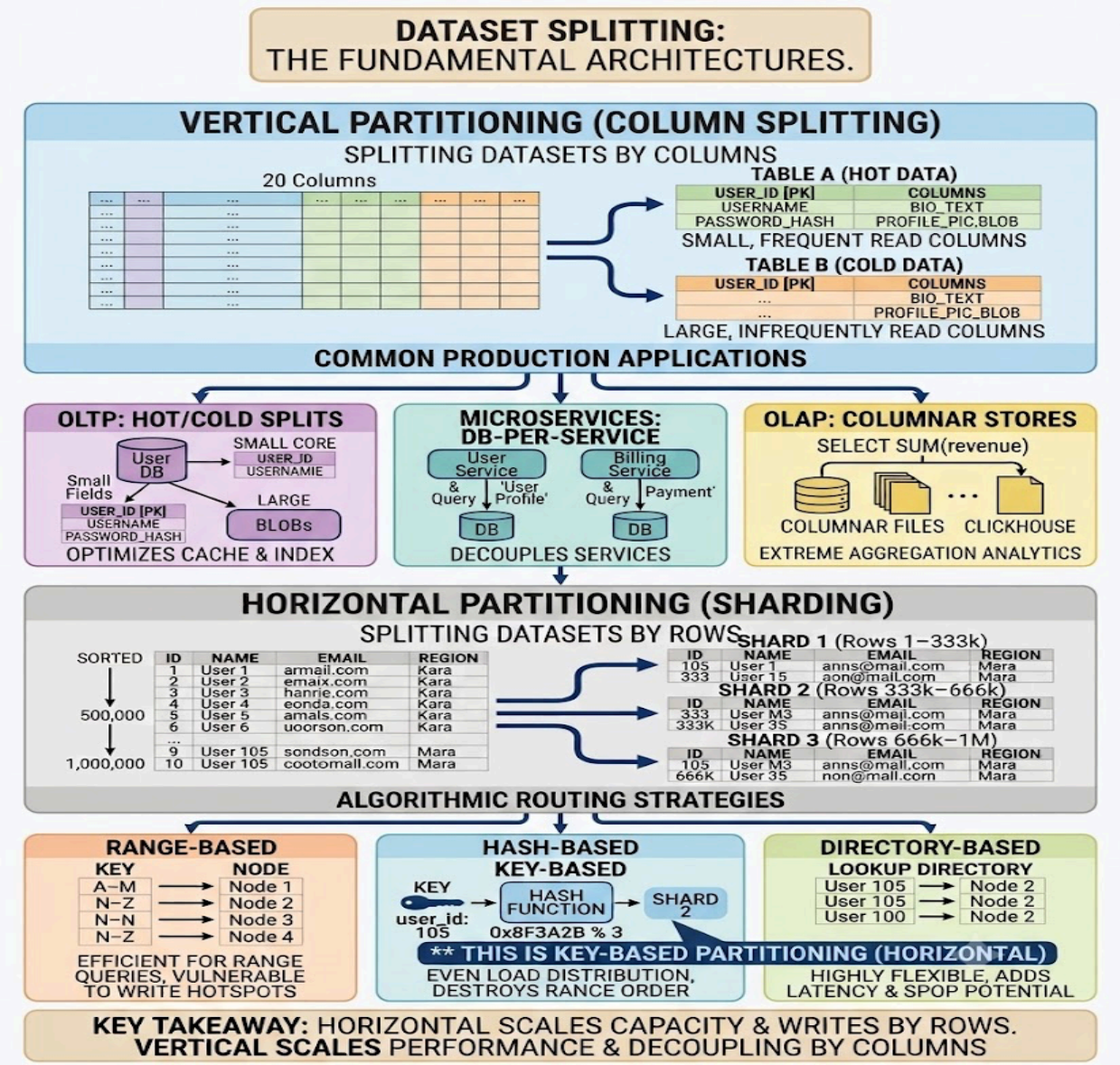
evaluating a database for a new system, since most systems spend far more time in the "no partition" state than the "partition" state.

Data Distribution: Partitioning & Consistent Hashing

Partitioning Strategies

The main methods of Data Partitioning are:
- Horizontal Partitioning/Sharding
- Vertical Partitioning

Categories for Horizontal and Vertical Partitioning -



Horizontal Partitioning, also called sharding, is the practice of splitting a large dataset across multiple machines so that no single machine has to store or serve all of it. Several strategies are common:

There are strategies to do horizontal partitioning -

Strategy	How It Works / Trade-off
Range-based	Keys are split into contiguous ranges (for example A-M, N-Z) and each range is assigned to a partition. Easy to reason about and supports efficient range queries, but can create hotspots if data or traffic is unevenly distributed across ranges.
Hash-based(Key based)	A hash function is applied to the key, and the result determines the partition. Distributes load evenly in most cases, but makes range queries across partitions inefficient.
Directory-based	A separate lookup service tracks which partition holds which key. Very flexible and supports rebalancing, but the lookup service becomes an additional component that must itself be reliable.

Vertical Partitioning, also called column splitting, is the practice of dividing a wide table or monolithic database into smaller, specialized tables with fewer columns, usually linked by a common primary key. Several strategies are common:

There are strategies to do vertical partitioning -

Strategy	How It Works / Trade-off
Hot/Cold Data Separation	Frequently accessed columns (like ID, username) are kept in a primary table, while large or rarely accessed columns (like bios, image BLOBs) are moved to a secondary table. Optimizes memory and index cache performance, but

	requires joins if the application needs to read all columns at once.
Domain-Based (Microservices)	Data is split along business domain boundaries (e.g., separating User tables from Billing tables into distinct databases). Decouples services and prevents monolithic bottlenecks, but makes cross-domain queries and distributed transactions highly complex.
Columnar Storage (OLAP)	Data is physically stored column-by-column on disk rather than row-by-row (e.g., Snowflake, ClickHouse). Enables incredibly fast analytical aggregations since only the required columns are read from disk, but makes transactional single-row inserts and updates very slow.

The Problem With Naive Hashing

A simple approach to hash-based partitioning is hash(key) modulo N, where N is the number of nodes. This works until N changes: adding or removing even one node changes the result of the modulo operation for almost every key, forcing nearly all data to be moved at once. In a live system, that is disruptive and slow.

How Consistent Hashing Works

Consistent hashing solves this problem. Both nodes and keys are hashed onto the same circular space, commonly visualized as a ring. A key is assigned to the first node found moving clockwise from the key's position on the ring. When a node is added or removed, only the keys between that node and its neighbor need to move; the rest of the ring is unaffected.

Virtual Nodes & Even Load Distribution

With only a few real nodes placed on the ring, load can still be uneven, since the gaps between nodes are not guaranteed to be equal. The common fix is to give each physical node many virtual positions on the ring, so that the ring is divided into many small, roughly equal segments. This smooths out load distribution and also means that when a physical node is added or removed, the resulting rebalancing work is spread across many other nodes rather than concentrated on one neighbor.

Replication Fundamentals

Why Replicate Data

Replication means keeping a copy of the same data on more than one machine. It is used for two main reasons: to survive the failure of any single machine without losing data, and to serve more read traffic than a single machine could handle by spreading reads across copies.

Synchronous vs Asynchronous Replication

Mode	How It Works / Trade-off
Synchronous	The write is only acknowledged to the client after it has been confirmed on the replica(s) as well as the primary. Guarantees replicas are always up to date, at the cost of higher write latency and reduced availability if a replica is slow or unreachable.
Asynchronous	The write is acknowledged as soon as the primary confirms it, and the update is sent to replicas afterward. Lower write latency and higher availability, but a replica can briefly lag behind, and data written just before a primary failure can be lost.

Leader-Follower vs Multi-Leader vs Leaderless

- Leader-follower: One node accepts all writes and propagates them to one or more read-only followers. Simple to reason about and avoids write conflicts, but the leader is a single point of write capacity and, if not handled carefully, a single point of failure.
- Multi-leader: More than one node can accept writes, typically one leader per data center or region, and leaders replicate to each other. This improves write availability and latency across regions, but introduces the possibility of conflicting writes to the same data that must be resolved.
- Leaderless: Any replica can accept a write, and the client (or a coordinator) writes to several replicas directly, later reconciling differences through techniques such as read repair. This maximizes availability but pushes more conflict-resolution responsibility onto the application or database logic.

Replication Trade-offs Through the CAP Lens

Replication choices are a direct, practical expression of the CAP trade-off described earlier. Synchronous, leader-based replication leans toward consistency at some cost to availability and latency. Asynchronous or leaderless replication leans toward availability and lower latency at

some cost to strict consistency. Neither is universally correct; the right choice depends on what the data represents and how the application will use it.

Common Pitfalls & Case Studies

Anonymized Example Failure Scenarios

- Scenario A - Split-brain during a partition: Two nodes in a leader-follower setup both believed they were the leader after a brief network partition, and both accepted conflicting writes, leading to data that had to be manually reconciled.
- Scenario B - Hot partition: A hash-based sharding scheme sent a disproportionate share of traffic to one shard because a small number of keys were accessed far more often than the rest, overloading that single shard while others sat idle.
- Scenario C - Silent replication lag: An application read from an asynchronous replica immediately after writing to the primary and did not see its own write, because replication had not yet caught up, producing confusing and hard-to-reproduce bug reports.

Lessons Learned

- Always be explicit about which consistency guarantees a data store provides, rather than assuming the strongest possible guarantee by default.
- Design sharding keys around actual access patterns, not just an even mathematical distribution, to avoid hot partitions.
- Treat replication lag as a normal, expected condition to design around, not an occasional bug to be surprised by.

Indexing vs. Replicas vs. Vertical Scaling/Splitting vs. Horizontal Sharding

Strategy	Primary Problem Solved	Best For	Main Limitation / Trade-off
Indexing	Slow SELECT query execution times (High disk I/O, full table scans)	Point lookups, range filters, sorting, and joins	Slows down INSERT/UPDATE writes; consumes RAM
Read Replicas	High Read Volume overwhelming the main DB CPU/Memory	Read-heavy workloads (e.g., 90% Reads / 10% Writes)	Does not help write throughput; subject to replication lag

Vertical Scaling	Running out of CPU, RAM, or Disk space on a single node	Quick fix without code or schema refactoring	Has a hard hardware limit; costs scale exponentially
Vertical Partitioning	Wide tables with huge text/binary columns or domain coupling	Isolating "hot" columns or decoupling microservices	Requires cross-table joins or application-level assembly
Horizontal Sharding	Write Volume or Data Size exceeds a single machine's physical limits	Multi-terabyte tables and extreme write throughput	High operational complexity; destroys cross-shard ACID transactions

GEM

- Find the gem link here and start asking the doubts.

Additional Resources

Public Tutorials & References

- Public engineering blog posts and papers on CAP theorem, consistent hashing, and replication strategies from well-known distributed database and cloud providers.
- "Designing Data-Intensive Applications" and similar widely used distributed systems references, for a deeper theoretical treatment of these topics.
- Gfg resources - https://www.geeksforgeeks.org/system-design/data-partitioning-techniques/